

Kysymys 9: Jos Sinulle annetaan seuraava linkki

(<https://archive.ics.uci.edu/ml/datasets/Horse+Colic> =>

<https://archive.ics.uci.edu/ml/machine-learning-databases/horse-colic/horse-colic.data>),

Miten saat linkin aineiston käyttöösi

Esimerkiksi Google Colabissa komennolla

```
# Lue lisää Pandas-paketin io-toiminnallisuuksista https://pandas.pydata.org/pandas-docs/stable/user\_guide/io.html
import pandas as pd

# Luetaan horse.data (sienien data)

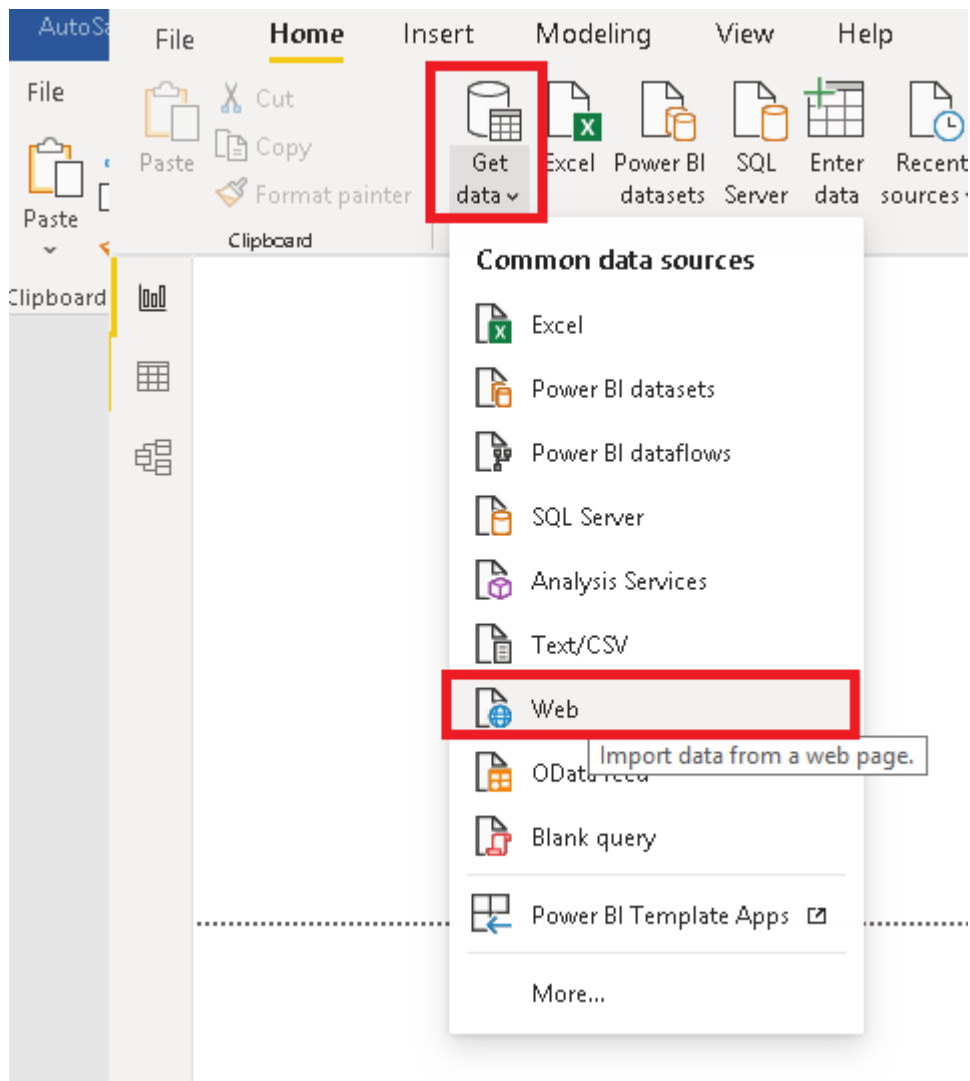
csv_path = 'https://archive.ics.uci.edu/ml/machine-learning-databases/horse-colic/horse-colic.data'

# Nimetään sarakkeet

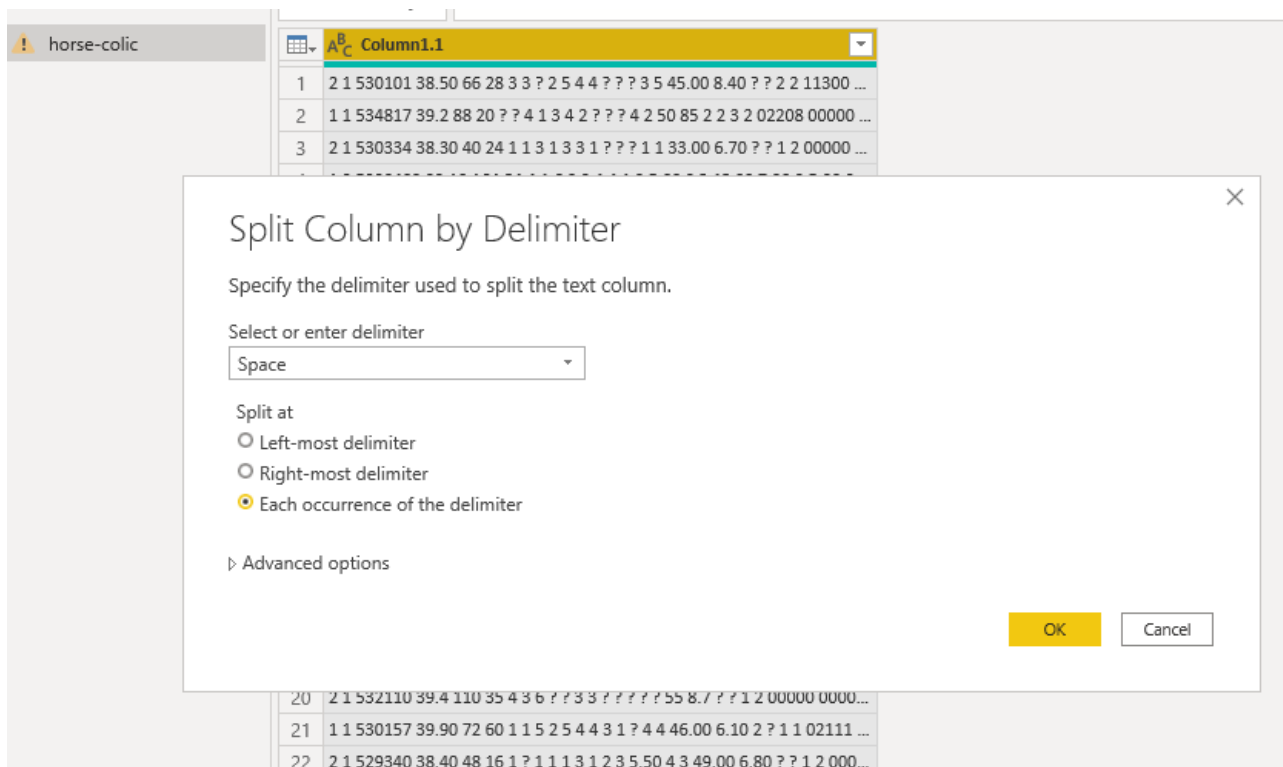
csv_columns = [
    "surgery", "age", "hospital-ID", "rectal-temp", "pulse", "resp-rate",
    "extremity-temp", "peripheral-pulse", "mucous-membranes",
    "capillary-refill-time", "pain", "peristalsis", "ab-distension",
    "nasogastric-tube", "nasogastric-reflux", "rectal-exam-feces",
    "abdomen", "packed-cell-vol", "total-protein", "abdominocentesis-appearance",
    "abdomcentesis-total-protein", "outcome", "surgical-lesion",
    "site-of-lesion", "type-of-lesion", "lesion-subtype", "lesion-ID",
    "cp_data"]

#luetaan data ja käytetään pilkkua erottimena
dataset = pd.read_csv(csv_path, names=csv_columns, delimiter=' ')
```

Power BI:ssä linkin aineiston saisi käyttöön menemällä 'Get Data' => 'Web' => johon antaisin URL: <https://archive.ics.uci.edu/ml/machine-learning-databases/horse-colic/horse-colic.data>



Jonka jälkeen erottelisin aineiston kentiksi "Split By Delimeter" => "Empty Space"



Minkä jälkeen nimeäisin kentät datan internet sivustolta löydetyn tiedoston perusteella jossa kerrotaan mitä kentät tarkoittavat

	A ^B _C Column1.1.1	1 ² ₃ Column1.1.2	1 ² ₃ Column1.1.3	A ^B _C Column1.1.4	A ^B _C Column1.1.5
1	2	1	530101	38.50	66
2	1	1	534817	39.2	88
3	2	1	530334	38.30	40
4	1	9	5290409	39.10	164
5	2	1	530255	37.30	104
6	2	1	528355	?	?
7	1	1	526802	37.90	48
8	1	1	529607	?	60
9	2	1	530051	?	80
10	2	9	5299629	38.30	90
11	1	1	528548	38.10	66
12	2	1	527927	39.10	72
13	1	1	528031	37.20	42
14	2	9	5291329	38.00	92
15	1	1	534917	38.2	76
16	1	1	530233	37.60	96
17	1	9	5301219	?	128
18	2	1	526639	37.50	48
19	1	1	5290481	37.60	64
20	2	1	532110	39.4	110
21	1	1	530157	39.90	72
22	2	1	529340	38.40	48
23	1	1	521681	38.60	42
24	1	9	534998	38.3	130
25	1	1	533692	38.1	60
26	2	1	529518	37.80	60
27	1	1	530526	38.30	72
28	1	1	528653	37.80	48
29	1	1	5279442	?	?
30	2	1	535415	37.7	48
31	2	1	529475	37.70	96

Mitä tekisin aineistolla

Loisin luokittelijan Google colabilla, jossa tarkastelisin eniten kenttään 'surgery' vaikuttaneita tekijöitä. Luokittelijan perusteella voidaan ennustaa sille annettujen tietojen perusteella sitä, pitääkö (todennäköisesti, luodun mallin tarkkuus oli yli 90%) hevosen oireita hoitaa leikkauksella vai voidaanko oireet hoitaa ilman leikkausta.

```

# Lue lisää Pandas-paketin io-toiminnallisuuksista https://pandas.pydata.org/pandas-docs/stable/user\_guide/io.html
import pandas as pd

# Luetaan horse.data (sienien data)

csv_path = 'https://archive.ics.uci.edu/ml/machine-learning-databases/horse-colic/horse-colic.data'

# Nimetään sarakkeet

csv_columns = [
    "surgery", "age", "hospital-ID", "rectal-temp", "pulse", "resp-rate",
    "extermity-temp", "peripheral-pulse", "mucous-membranes",
    "capillary-refill-time", "pain", "peristalsis", "ab-distension",
    "nasogastric-tube", "nasogastric-reflux", "rectal-exam-feces",
    "abdomen", "packed-cell-vol", "total-protein", "abdominocentesis-appearance",
    "abdomcentesis-total-protein", "outcome", "surgical-lesion",
    "site-of-lesion", "type-of-lesion", "lesion-subtype", "lesion-ID",
    "cp_data"]

#luetaan data ja käytetään pilkkua erottimena
dataset = pd.read_csv(csv_path, names=csv_columns, delimiter=' ')

# Muodotetaan csv-tiedosto
dataset.to_csv('UCI_horse_colab.csv')

```

14] dataset

	surgery	age	hospital-ID	rectal-temp	pulse	resp-rate	extermity-temp	peripheral-pulse	mucous-membranes	capillary-refill-time	pain	peristalsis	ab-distension	nasogastric tube
0	2	1	530101	38.50	66	28	3	3	?	2	5	4	4	
1	1	1	534817	39.2	88	20	?	?	4	1	3	4	2	
2	2	1	530334	38.30	40	24	1	1	3	1	3	3	1	

```

[16] # Jaetaan datajoukko kahteen osaan
data = dataset.sample(frac=0.80, random_state=786)
data_unseen = dataset.drop(data.index)
data.reset_index(inplace=True, drop=True)
data_unseen.reset_index(inplace=True, drop=True)
print('Mallinnukseen käytettävä data: ' + str(data.shape))
print('Ennakointiin käytettävä data: ' + str(data_unseen.shape))

```

 Mallinnukseen käytettävä data: (240, 28)
 Ennakointiin käytettävä data: (60, 28)

```

# Asennetaan PyCaret
!pip install pycaret

```

```
[17] # Ladataan paketit, joita PyCaret tarvitsee
import numpy as np
import warnings
```

```
[18] # Ladataan PyCaret-paketista kaikki luokittelijavaihtoehdot
from pycaret.classification import *
```

```
▶ # Muodostetaan luokittelijat
clf=setup(data=data,target='surgery')
```

📁 Setup Successfully Completed!

	Description	Value
0	session_id	3612
1	Target Type	Binary
2	Label Encoded	1: 0, 2: 1
3	Original Data	(240, 28)
4	Missing Values	False
5	Numeric Features	2
6	Categorical Features	25
7	Ordinal Features	False
8	High Cardinality Features	False
9	High Cardinality Method	None
10	Sampled Data	(240, 28)
11	Transformed Train Set	(167, 438)
12	Transformed Test Set	(73, 438)
13	Numeric Imputer	mean
14	Categorical Imputer	constant
15	Normalize	False
16	Normalize Method	None
17	Transformation	False
18	Transformation Method	None
19	PCA	False

```
# Vertaa luokittelijoita
compare_models()
```

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC	TT (Sec)
0	Extreme Gradient Boosting	0.8449	0.8819	0.7524	0.8589	0.7857	0.6697	0.6860	0.1799
1	CatBoost Classifier	0.8390	0.8919	0.7381	0.8606	0.7836	0.6588	0.6740	2.5475
2	Gradient Boosting Classifier	0.8386	0.8764	0.7810	0.8327	0.7971	0.6646	0.6755	0.1623
3	Extra Trees Classifier	0.8261	0.8730	0.6738	0.8869	0.7461	0.6219	0.6498	0.2558
4	Light Gradient Boosting Machine	0.8210	0.8817	0.7524	0.8204	0.7658	0.6260	0.6430	0.0252
5	Random Forest Classifier	0.8132	0.8661	0.6548	0.8669	0.7348	0.5966	0.6187	0.1154
6	Decision Tree Classifier	0.7842	0.7800	0.7500	0.7530	0.7345	0.5562	0.5704	0.0129
7	Ridge Classifier	0.7842	0.0000	0.7500	0.7581	0.7392	0.5599	0.5721	0.0144
8	Ada Boost Classifier	0.7728	0.7674	0.6881	0.7638	0.7092	0.5254	0.5412	0.1156
9	K Neighbors Classifier	0.7000	0.7375	0.5810	0.6414	0.6069	0.3655	0.3692	0.0108
10	Logistic Regression	0.5993	0.6986	0.0000	0.0000	0.0000	0.0000	0.0000	0.0222
11	SVM - Linear Kernel	0.5640	0.0000	0.2000	0.0824	0.1167	0.0000	0.0000	0.0123
12	Linear Discriminant Analysis	0.5438	0.5536	0.5476	0.4485	0.4880	0.0871	0.0867	0.0403
13	Naive Bayes	0.5044	0.6740	0.8976	0.4453	0.5939	0.1179	0.1406	0.0105
14	Quadratic Discriminant Analysis	0.4493	0.5336	0.9571	0.4205	0.5829	0.0560	0.0940	0.0238

```
XGBClassifier(base_score=0.5, booster='gbtree', colsample_bylevel=1,
               colsample_bynode=1, colsample_bytree=1, gamma=0,
               learning_rate=0.1, max_delta_step=0, max_depth=3,
               min_child_weight=1, missing=None, n_estimators=100, n_jobs=-1,
               nthread=None, objective='binary:logistic', random_state=3612,
               reg_alpha=0, reg_lambda=1, scale_pos_weight=1, seed=None,
               silent=None, subsample=1, verbosity=0)
```

```
# Lisäinformaatiota malleista löytyy, kun hakee Googlestä Reference-sarakkeessa mainittua
models()
```

ID	Name	Reference	Turbo
lr	Logistic Regression	sklearn.linear_model.LogisticRegression	True
knn	K Neighbors Classifier	sklearn.neighbors.KNeighborsClassifier	True
nb	Naive Bayes	sklearn.naive_bayes.GaussianNB	True
dt	Decision Tree Classifier	sklearn.tree.DecisionTreeClassifier	True
svm	SVM - Linear Kernel	sklearn.linear_model.SGDClassifier	True
rbfsvm	SVM - Radial Kernel	sklearn.svm.SVC	False
gpc	Gaussian Process Classifier	sklearn.gaussian_process.GPC	False
mlp	MLP Classifier	sklearn.neural_network.MLPClassifier	False
ridge	Ridge Classifier	sklearn.linear_model.RidgeClassifier	True
rf	Random Forest Classifier	sklearn.ensemble.RandomForestClassifier	True

```
# Listataan mallien lyhenteet
model_metadata = models()
model_metadata['Name']
```

```
ID
lr                Logistic Regression
knn               K Neighbors Classifier
nb               Naive Bayes
dt               Decision Tree Classifier
svm              SVM - Linear Kernel
rbfsvm           SVM - Radial Kernel
gpc              Gaussian Process Classifier
mlp              MLP Classifier
ridge            Ridge Classifier
rf               Random Forest Classifier
qda              Quadratic Discriminant Analysis
ada              Ada Boost Classifier
gbc              Gradient Boosting Classifier
lda              Linear Discriminant Analysis
et               Extra Trees Classifier
xgboost          Extreme Gradient Boosting
lightgbm         Light Gradient Boosting Machine
catboost         CatBoost Classifier
Name: Name, dtype: object
```

Luodaan luokittelijat, jotka ovat merkitty keltaisella `compare_models()` tuloksissa. HUOM: valitsin luokittelijoiksi aiemmin colab tiedostossa olleet luokittelijat, koska ne kaikki olivat top-6 luokittelijoiden joukossa, joten riittävän hyviä

```
lightgbm = create_model('lightgbm')
```

	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	0.6471	0.7286	0.2857	0.6667	0.4000	0.2031	0.2398
1	0.8235	0.9143	0.7143	0.8333	0.7692	0.6277	0.6326
2	0.9412	0.9714	0.8571	1.0000	0.9231	0.8759	0.8827
3	0.8235	0.7857	0.5714	1.0000	0.7273	0.6107	0.6630
4	0.7647	0.9000	0.8571	0.6667	0.7500	0.5342	0.5494
5	0.9412	0.9714	0.8571	1.0000	0.9231	0.8759	0.8827
6	0.7059	0.8286	0.7143	0.6250	0.6667	0.4056	0.4085
7	0.9375	1.0000	0.8333	1.0000	0.9091	0.8621	0.8704
8	0.6875	0.7500	0.8333	0.5556	0.6667	0.3939	0.4229
9	0.9375	0.9667	1.0000	0.8571	0.9231	0.8710	0.8783
Mean	0.8210	0.8817	0.7524	0.8204	0.7658	0.6260	0.6430
SD	0.1098	0.0957	0.1899	0.1692	0.1581	0.2303	0.2230


```
[24] catboost = create_model('catboost')
```

	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	0.6471	0.6571	0.4286	0.6000	0.5000	0.2388	0.2469
1	0.8235	0.9286	0.7143	0.8333	0.7692	0.6277	0.6326
2	0.8824	0.9571	0.8571	0.8571	0.8571	0.7571	0.7571
3	0.8235	0.8429	0.5714	1.0000	0.7273	0.6107	0.6630
4	0.8824	0.9286	0.7143	1.0000	0.8333	0.7463	0.7715
5	0.8824	0.9714	0.7143	1.0000	0.8333	0.7463	0.7715
6	0.8235	0.9000	0.7143	0.8333	0.7692	0.6277	0.6326
7	0.9375	1.0000	0.8333	1.0000	0.9091	0.8621	0.8704
8	0.7500	0.7500	0.8333	0.6250	0.7143	0.5000	0.5164
9	0.9375	0.9833	1.0000	0.8571	0.9231	0.8710	0.8783
Mean	0.8390	0.8919	0.7381	0.8606	0.7836	0.6588	0.6740
SD	0.0840	0.1054	0.1506	0.1424	0.1159	0.1781	0.1781

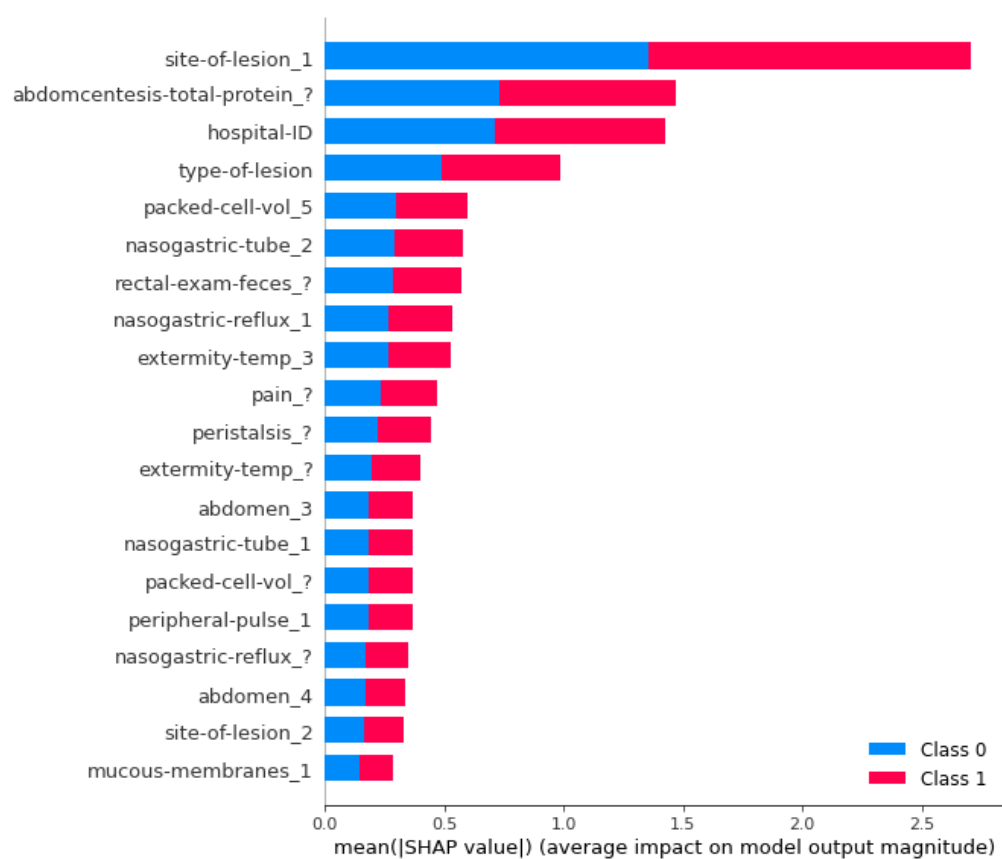
```
[25] dt = create_model('dt')
```

	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	0.5882	0.5857	0.5714	0.5000	0.5333	0.1678	0.1690
1	0.9412	0.9286	0.8571	1.0000	0.9231	0.8759	0.8827
2	0.8824	0.8786	0.8571	0.8571	0.8571	0.7571	0.7571
3	0.7647	0.7786	0.8571	0.6667	0.7500	0.5342	0.5494
4	0.8824	0.8786	0.8571	0.8571	0.8571	0.7571	0.7571
5	0.7647	0.7143	0.4286	1.0000	0.6000	0.4688	0.5533
6	0.7059	0.6857	0.5714	0.6667	0.6154	0.3796	0.3825
7	0.7500	0.7667	0.8333	0.6250	0.7143	0.5000	0.5164
8	0.6250	0.6333	0.6667	0.5000	0.5714	0.2500	0.2582
9	0.9375	0.9500	1.0000	0.8571	0.9231	0.8710	0.8783
Mean	0.7842	0.7800	0.7500	0.7530	0.7345	0.5562	0.5704
SD	0.1180	0.1196	0.1701	0.1772	0.1418	0.2386	0.2366

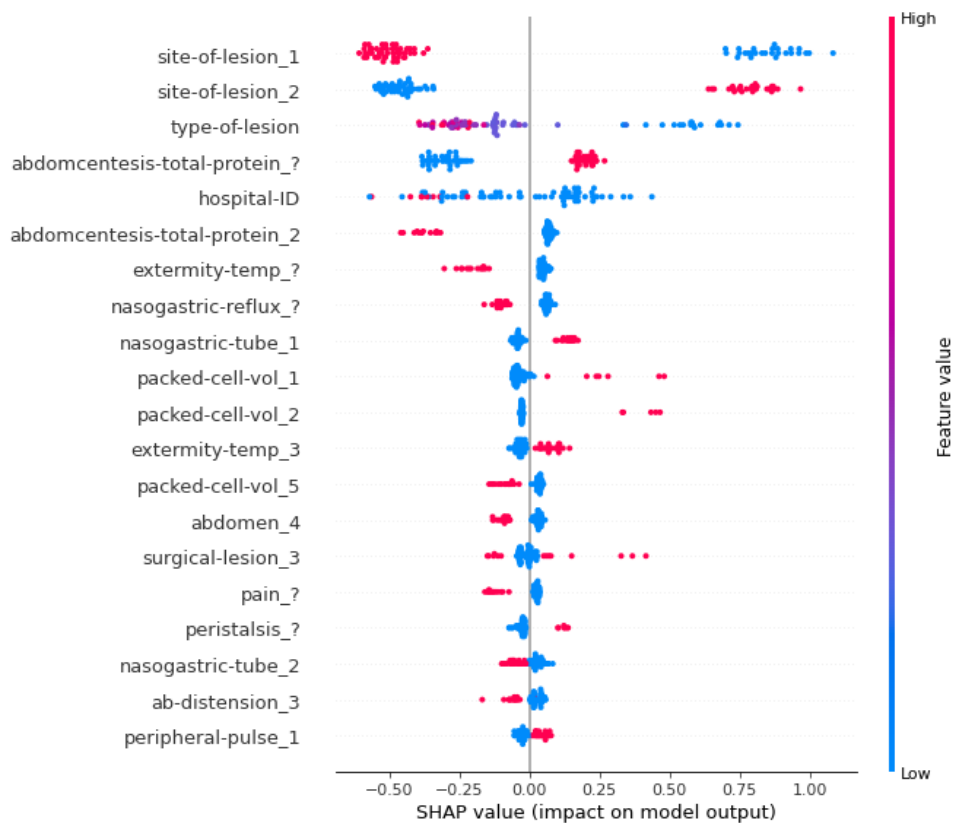
Tulkitaan luokittelijoihin vaikuttavia tekijöitä ennen kuin "tuunataan"/viritetään niitä

```
# Lue lisää https://shap.readthedocs.io/en/latest/  
!pip install shap
```

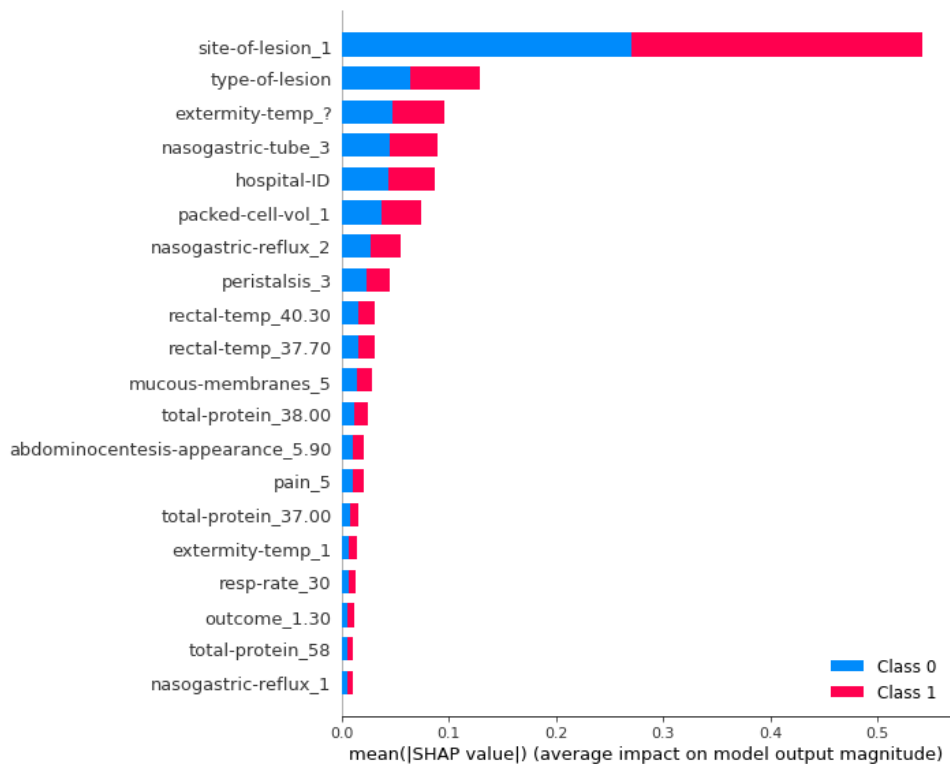
```
interpret_model(lightgbm)
```



`interpret_model(catboost)`



`interpret_model(dt)`



"Tuunataan"/viritetään luodut mallit hyperparametrien suhteen

```
[30] tuned_lightgbm = tune_model(lightgbm)
```

	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	0.5882	0.6214	0.2857	0.5000	0.3636	0.0916	0.0994
1	0.8235	0.8429	0.7143	0.8333	0.7692	0.6277	0.6326
2	0.8235	0.9571	0.8571	0.7500	0.8000	0.6434	0.6480
3	0.8235	0.8143	0.5714	1.0000	0.7273	0.6107	0.6630
4	0.8235	0.9000	0.8571	0.7500	0.8000	0.6434	0.6480
5	0.8824	0.8429	0.7143	1.0000	0.8333	0.7463	0.7715
6	0.8824	0.8857	0.8571	0.8571	0.8571	0.7571	0.7571
7	0.9375	1.0000	0.8333	1.0000	0.9091	0.8621	0.8704
8	0.6875	0.7667	0.8333	0.5556	0.6667	0.3939	0.4229
9	0.9375	1.0000	1.0000	0.8571	0.9231	0.8710	0.8783
Mean	0.8210	0.8631	0.7524	0.8103	0.7649	0.6247	0.6391
SD	0.1033	0.1091	0.1899	0.1677	0.1528	0.2208	0.2193

```
[31] tuned_catboost = tune_model(catboost)
```



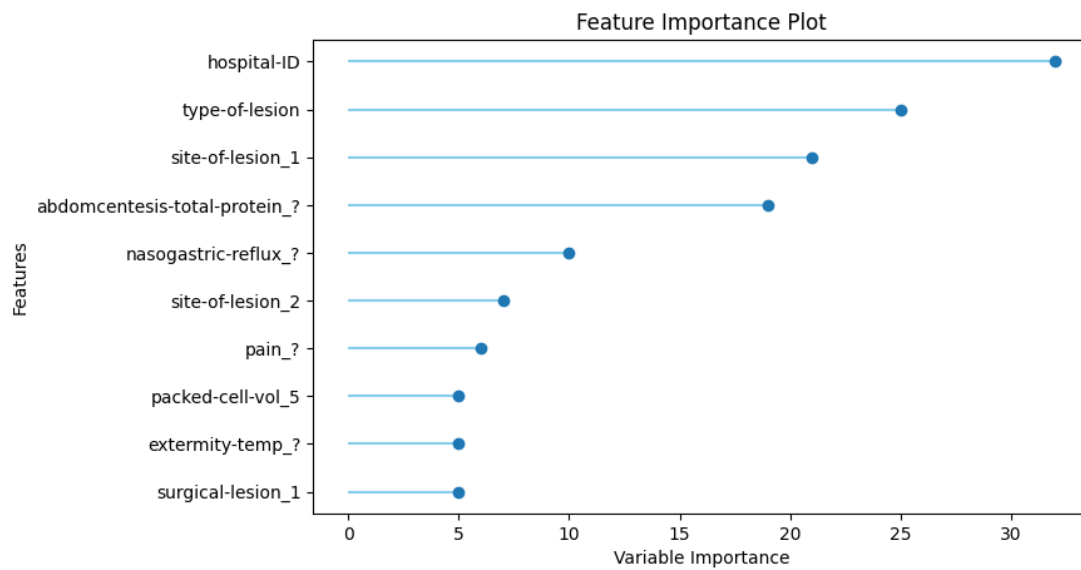
	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	0.6471	0.6571	0.4286	0.6000	0.5000	0.2388	0.2469
1	0.8235	0.9429	0.7143	0.8333	0.7692	0.6277	0.6326
2	0.8824	0.8857	0.8571	0.8571	0.8571	0.7571	0.7571
3	0.8235	0.8143	0.5714	1.0000	0.7273	0.6107	0.6630
4	0.8824	0.9714	0.7143	1.0000	0.8333	0.7463	0.7715
5	0.8824	0.9429	0.7143	1.0000	0.8333	0.7463	0.7715
6	0.8235	0.9143	0.8571	0.7500	0.8000	0.6434	0.6480
7	0.8125	1.0000	0.5000	1.0000	0.6667	0.5556	0.6202
8	0.7500	0.7500	0.8333	0.6250	0.7143	0.5000	0.5164
9	0.9375	0.9667	1.0000	0.8571	0.9231	0.8710	0.8783
Mean	0.8265	0.8845	0.7190	0.8523	0.7624	0.6297	0.6506
SD	0.0774	0.1049	0.1686	0.1461	0.1131	0.1665	0.1658

```
[32] tuned_dt = tune_model(dt)
```

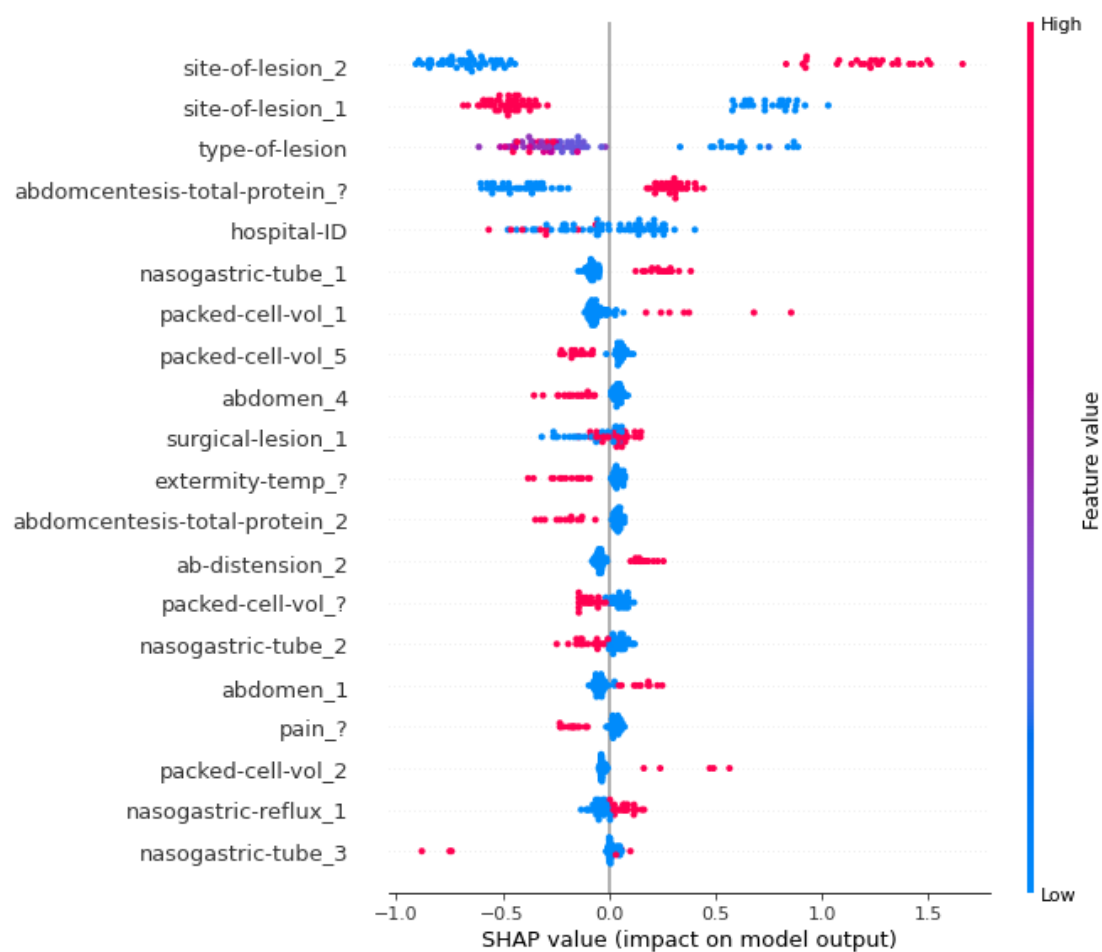


	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	0.5294	0.5143	0.4286	0.4286	0.4286	0.0286	0.0286
1	0.7647	0.8643	0.7143	0.7143	0.7143	0.5143	0.5143
2	0.6471	0.8357	0.7143	0.5556	0.6250	0.3014	0.3099
3	0.8235	0.7857	0.5714	1.0000	0.7273	0.6107	0.6630
4	0.7647	0.8929	0.7143	0.7143	0.7143	0.5143	0.5143
5	0.8235	0.8214	0.7143	0.8333	0.7692	0.6277	0.6326
6	0.6471	0.8286	0.5714	0.5714	0.5714	0.2714	0.2714
7	0.9375	0.9833	1.0000	0.8571	0.9231	0.8710	0.8783
8	0.6250	0.7750	0.8333	0.5000	0.6250	0.2941	0.3333
9	0.8125	0.8333	0.6667	0.8000	0.7273	0.5862	0.5919
Mean	0.7375	0.8135	0.6929	0.6975	0.6825	0.4620	0.4738
SD	0.1159	0.1143	0.1469	0.1714	0.1244	0.2271	0.2297

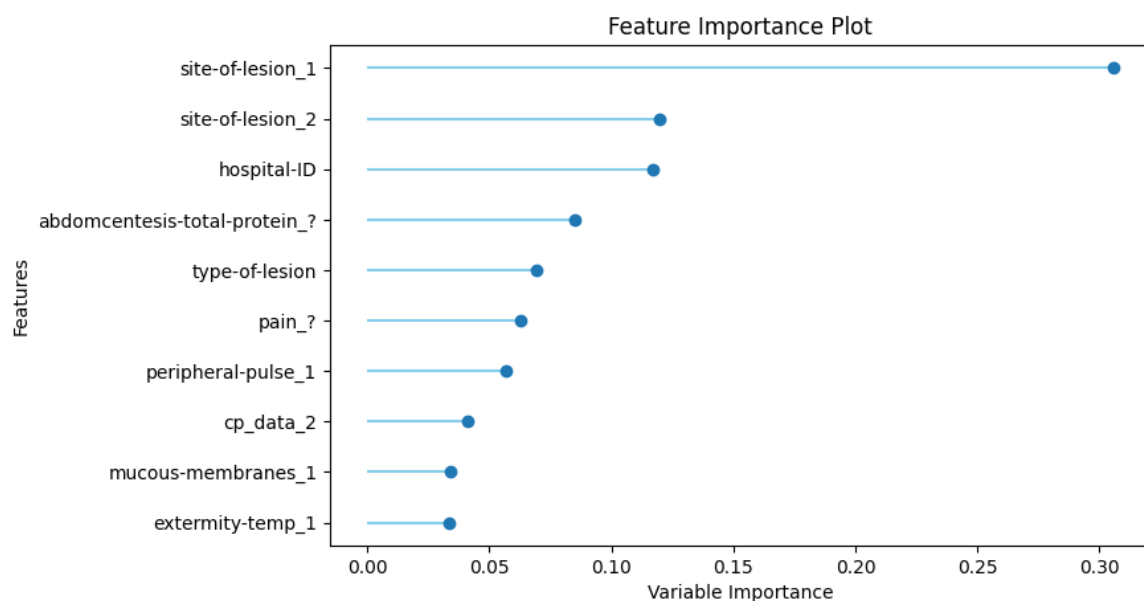
```
[33] plot_model(tuned_lightgbm, plot='feature')
```



```
interpret_model(tuned_catboost)
```



```
plot_model(tuned_dt, plot='feature')
```



[illegible]

predict_model(tuned_catboost)														
	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC						
0	CatBoost Classifier	0.7945	0.8101	0.6667	0.8	0.7273	0.5646	0.5707						
	hospital-ID	type-of-lesion	age_1	age_9	rectal-temp_35.4	rectal-temp_36.00	rectal-temp_36.10	rectal-temp_36.40	rectal-temp_36.60	rectal-temp_36.80	rectal-temp_37.00	rectal-temp_37.1	rectal-temp_37.10	
0	529628.0	300.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
1	533902.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
2	535314.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
3	535163.0	4124.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
4	534053.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	
68	5290481.0	4205.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
69	5290482.0	4205.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
70	529493.0	5205.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
71	529796.0	2124.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
72	527758.0	3124.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
73 rows x 14 columns														

```
[38] predict_model(tuned_dt);
```

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	Decision Tree Classifier	0.726	0.7488	0.4333	0.8125	0.5652	0.3912	0.4324

Viimeistellään valittu luokittelija koskemaan koko datajoukkoa ja kokeillaan luokittelua siihen data_unseen-nimiseen datajoukkoon, jota ei käytetty luokittelijan muodostamisessa

```
[39] final_lightgbm = finalize_model(tuned_lightgbm)
```

```
[40] print(final_lightgbm)
```

```
LGBMClassifier(boosting_type='gbdt', class_weight=None, colsample_bytree=1.0,
importance_type='split', learning_rate=0.1, max_depth=80,
min_child_samples=20, min_child_weight=0.001, min_split_gain=0.4,
n_estimators=70, n_jobs=-1, num_leaves=100, objective=None,
random_state=3612, reg_alpha=0.7, reg_lambda=0.3, silent=True,
subsample=1.0, subsample_for_bin=200000, subsample_freq=0)
```

```
[41] unseen_predictions = predict_model(final_lightgbm, data=data_unseen)
unseen_predictions.head()
```

	surgery	age	hospital- ID	rectal- temp	pulse	resp- rate	extermity- temp	peripheral- pulse	mucous- membranes	capillary- refill- time	pain
0	1	9	5290409	39.10	164	84	4	1	6	2	2
1	2	1	528355	?	?	?	2	1	3	1	2
2	2	1	526639	37.50	48	24	?	?	?	?	?
3	1	1	530157	39.90	72	60	1	1	5	2	5
4	1	9	534998	38.3	130	60	?	3	?	1	2

```
from pycaret.utils import check_metric
unseen_predictions.rename(columns = {'surgery':'Origin'}, inplace = True)
check_metric(unseen_predictions.Origin, unseen_predictions.Label, 'Accuracy')
```

0.9167

Tallennetaan luokittelija

+ Code

+ Text

```
[47] save_model(final_lightgbm, 'Horse_luokittelija')
```

Transformation Pipeline and Model Succesfully Saved

Otetaan talteen muokattu datasetti

The screenshot shows a JupyterLab environment. On the left, the 'Files' panel displays a directory structure with files like 'catboost_info', 'sample_data', 'Horse_luokittelija.pkl', 'Mushroom_luokittelija.pkl', 'UCI_horse_colab.csv', and 'logs.log'. The file 'UCI_horse_colab.csv' is highlighted with a red box. A context menu is open over this file, with the 'Download' option also highlighted by a red box. The main area shows a code editor with two cells. The first cell contains a list of parameters: `[40] n_estimators=70, n_job random_state=3612, reg subsample=1.0, subsamp`. The second cell contains a prediction call: `[41] unseen_predictions = predict_model(fi unseen_predictions.head())`. Below the code, a table of data is displayed with columns: 'surgery', 'age', 'hospital-ID', and 'rectal-temp'. The table has 6 rows of data. At the bottom, a third code cell is partially visible, showing `from pycaret.utils import check_metric` and `unseen_predictions.rename(columns = {`.

Files Panel:

- catboost_info
- sample_data
- Horse_luokittelija.pkl
- Mushroom_luokittelija.pkl
- UCI_horse_colab.csv**
- logs.log

Context Menu:

- Download**
- Delete file
- Rename file
- Copy path
- Refresh

Code Cell [40]:

```
[40] n_estimators=70, n_job  
      random_state=3612, reg  
      subsample=1.0, subsamp
```

Code Cell [41]:

```
[41] unseen_predictions = predict_model(fi  
      unseen_predictions.head())
```

Data Table:

surgery	age	hospital-ID	rectal-temp
9		5290409	39.10
1		528355	?
1		526639	37.50
1		530157	39.90
4	1	534998	38.3

Code Cell:

```
from pycaret.utils import check_metric  
unseen_predictions.rename(columns = {  
check_metric(unseen_predictions.Orig
```

0.9167

Otetaan talteen luotu luokittelija myöhempää käyttöä varten

The screenshot shows a file explorer window on the left and a code editor on the right. The file explorer displays a directory structure with files like `catboost_info`, `sample_data`, `Horse_luokittelija.pkl`, `Mushroom_luokittelija.pkl`, `UCI_horse_colab.csv`, and `logs.log`. The file `Horse_luokittelija.pkl` is selected, and a context menu is open over it, showing options like `Download`, `Delete file`, `Rename file`, `Copy path`, and `Refresh`. The `Download` option is highlighted. The code editor on the right shows Python code for saving a model, with the following lines visible:

```
[40] n_esti
      random
      subsam

[41] unseen_predictions =
      unseen_predictions.he
```

Below the code editor, there is a table with the following data:

				hosp
				52
				5
				5
3	1	1	5	
4	1	9	5	